

Lecture 01 : Introduction to Low Level Design

Low Level Design (LLD) woh process hai jisme hum **multiple DSA concepts** aur **Object-Oriented principles** ko combine karke ek **complete, scalable, maintainable** aur **reusable application** ka structure banate hain.



What is Low Level Design?

DSA (Data Structures & Algorithms) mein hum **isolated problems** ko solve karte hain. Jaise array mein element search karna, ya array ko sort karna. Har ek problem ka ek specific algorithm hota hai — Linear Search, Binary Search, Quick Sort, Merge Sort, etc.

Lekin jab hum **multiple DSA concepts** ko lekar ek **poori application** build karte hain, us process ko hum **Low Level Design (LLD)** kehte hain. LLD sirf algorithms nahi, balki application ke **objects, entities, unke beech ke relationships, data security, scalability, aur maintainability** par focus karta hai.

IMPORTANT DISTINCTION

DSA = Isolated Problem Solving | LLD = Complete Application Architecture using DSA + Design Principles

The Ride Booking Problem: Two Perspectives

Ek ride booking platform "**Quick Ride**" banani hai. Is problem ko do developers alag tarike se approach karte hain. In dono approaches se LLD aur DSA ke beech ka farq crystal clear ho jata hai.

Approach 1: The DSA-Only Mindset

CAUTION: INCOMPLETE THINKING

Developer jo sirf DSA jaanta hai, woh directly algorithms pe jump kar deta hai:

- **Shortest Path Problem:** City ke intersections ko **nodes** aur roads ko **edges** maan ke ek **graph** banaya. Dijkstra algorithm se shortest route nikaala.
- **Nearest Rider Problem:** Har user ke around ke riders ko ek **Min Heap (Priority Queue)** mein daala. Pop karne pe closest rider mil jata hai.

Manager ka feedback: "Ye sab toh algorithm hai. Application kahan hai? Objects kahan hain? Relationships kahan hain? Data security, notifications, payment gateway, scalability — ye sab kahan hai?"

Approach 2: The LLD Mindset

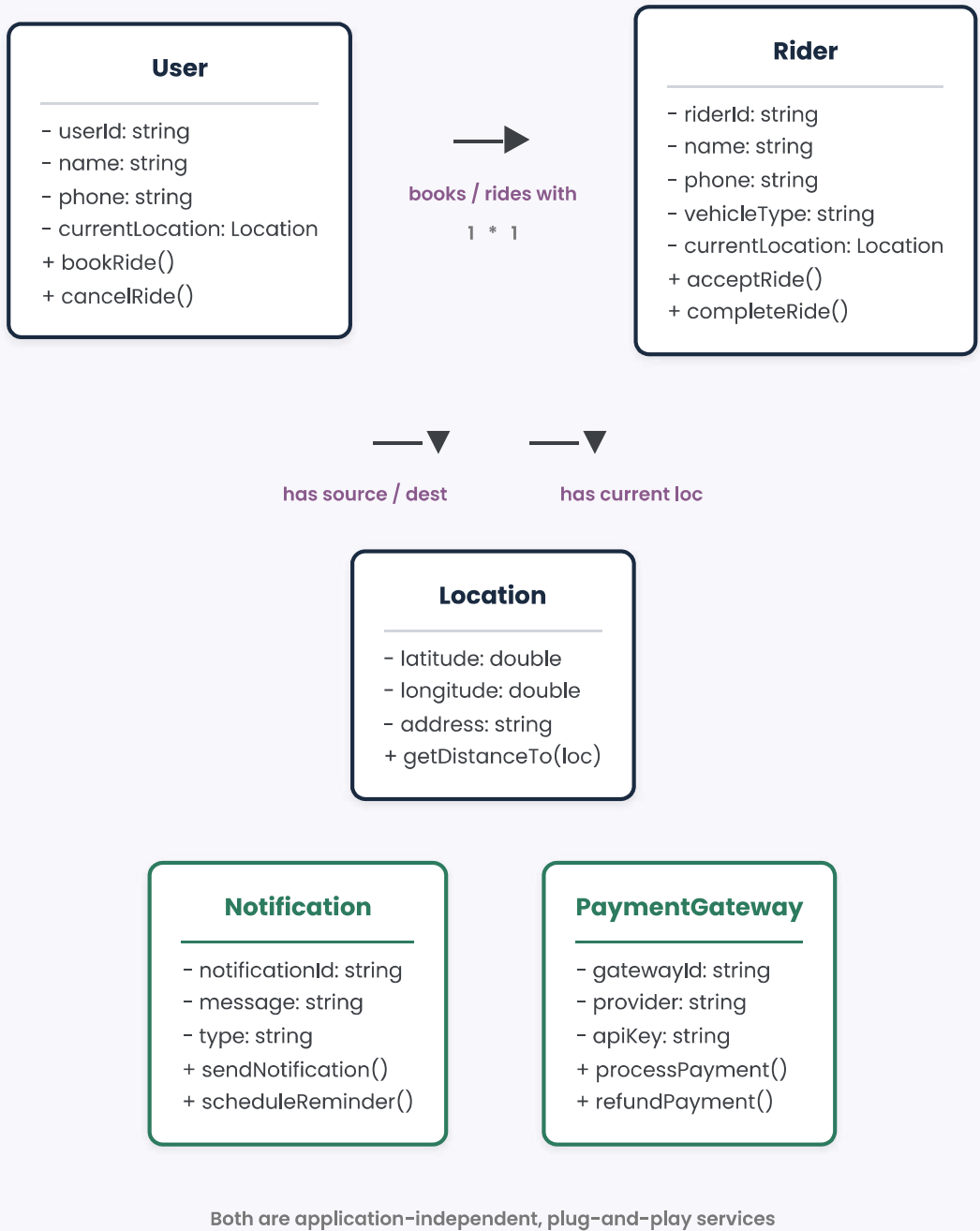
WORKFLOW: THE LLD PROCESS

Developer jo LLD jaanta hai, woh pehle application ka **structure (skeleton)** banata hai, uske baad DSA apply karta hai:

1. **Entities Identify karo:** User, Rider, Location, Notification, Payment — sabse pehle yeh objects define hote hain.
2. **Relationships establish karo:** User ka Rider se kya relation hai? Location ka User aur Rider se kya relation hai?
3. **Data Security socho:** Ride complete hone ke baad User ko Rider ka phone number nahi dikhna chahiye. Reverse bhi nahi.
4. **Scalability plan karo:** Millions of users aa jaye toh application kaise sustain karegi?
5. **Reusability ensure karo:** Notification system aur payment gateway aise design karo ki kisi bhi dusri application mein plug-and-play ho sakein.
6. **Ab DSA lagao:** Jab structure ready ho jaye, tab shortest path aur nearest rider algorithms implement karo.

Class Diagram: Quick Ride Application

DIAGRAM: CORE ENTITIES & RELATIONSHIPS



The Three Pillars of LLD

Har LLD decision teen core principles pe based hota hai. In teeno ko samajhna is absolutely critical for any system design interview ya real-world development.

1. Scalability

CRITICAL CONCEPT

Scalability yeh measure karti hai ki tumhari application **millions of users** ko ek saath handle kar paayegi ya nahi. Iska matlab sirf traffic handle karna nahi hai — balki:

- Naya feature integrate karna kitna easy hai?
- Codebase ko bina tode expand kiya ja sakta hai?
- Application performance degrade nahi hoti jab user base badhta hai?

2. Maintainability

TAKEAWAY

Maintainability ensure karti hai ki tumhara code **easily debuggable** ho aur naye features add karne pe **purane features break na hon**.

- Naya feature daalne se 4 purane features mein bug nahi aana chahiye.
- Code aisa likha ho ki koi bhi developer aake samajh sake aur modify kar sake.
- Bug identification aur fixing mein minimum time lage.

Zero bugs impossible hai, but minimum bugs aur easy debugging — yeh maintainability ka goal hai.

3. Reusability

INTUITION: PLUG-AND-PLAY ARCHITECTURE

Reusability ka matlab hai ki tumhara code **tightly coupled** nahi hona chahiye kisi specific application se. Usko uthake kisi dusri application mein daala jaye toh bina major changes ke chal jaaye.

Examples:

- **Notification Service:** Quick Ride mein jo notification system banaoge, woh Zomato, Swiggy ya kisi bhi app mein reuse ho sakta hai.
- **Payment Gateway:** Payment integration ek baar achhe se design kar lo, toh har nayi app mein wahi module plug kar do.
- **Rider Mapping Algorithm:** Nearest delivery partner assign karna — ye algorithm Zomato, Swiggy, Amazon, Blinkit sab mein same concept pe based hai. Application-specific nahi hona chahiye.

CAUTION: TIGHT COUPLING

Agar tumhara rider-mapping algorithm sirf Quick Ride ke specific data structures pe dependent hai, toh woh **tightly coupled** hai. Usse Zomato mein use karna mushkil ho jayega. LLD mein hum hamesha **loosely coupled, highly cohesive** modules design karte hain.

LLD vs HLD vs DSA: The Complete Picture

Bahut se beginners LLD aur HLD mein confuse hote hain. Dono alag-alag cheezein hain, aur dono ka alag purpose hai. Niche ka table is confusion ko permanently clear kar dega.

Aspect	DSA (Data Structures & Algorithms)	LLD (Low Level Design)	HLD (High Level Design)
Focus	Isolated problems solve karna	Code ka structure, classes, objects, relationships	System architecture, servers, databases
Scope	Single algorithm — search, sort, shortest path	Complete application ka low-level blueprint	Infrastructure, deployment, scaling strategy
Output	Optimized function / procedure	Class diagrams, sequence diagrams, C++ code structure	System design diagrams, architectural flow
Code	Heavy coding	Heavy coding (C++ classes, interfaces)	Almost zero code — architecture focused
Key Decisions	Time & space complexity	Scalability, Maintainability, Reusability	Tech stack, DB choice, server scaling, cost
Interview	Coding rounds	Machine coding / LLD rounds	System design rounds

What is NOT LLD? (The HLD Scope)

STRUCTURE: HLD DECISIONS

Yeh woh decisions hain jo **High Level Design** mein aate hain, LLD mein nahi:

- **Tech Stack:** Application Java Spring Boot mein banegi ya Node.js mein? Ye HLD ka decision hai.
- **Database Choice:** SQL (PostgreSQL, MySQL) use karna hai ya NoSQL (MongoDB, Cassandra)? Ya hybrid model?
- **Server Scaling:** Application ko servers pe deploy karne ke baad, millions of users aane pe servers kaise expand honge? Auto-scaling strategies?
- **Cost Optimization:** AWS servers ka cost kaise minimize kiya jaye? Traffic kam hone pe servers automatically scale down hone chahiye taaki extra cost na lage.

INTERVIEW TIP

HLD interview mein tum **almost zero code** likhte ho. Tum sirf boxes aur arrows se system architecture explain karte ho. LLD interview mein tum actual **class diagrams** aur **C++ code** likhke dikhte ho ki classes kaise interact karenge.

The Ultimate Analogy

KEY INSIGHT: THE BRAIN & SKELETON

*"If **DSA** is the **brain** of an application, then **LLD** is the **skeleton**."*

Brain (DSA) sochta hai — algorithms, logic, optimization. Skeleton (LLD) structure deta hai — kaunsa part kahan judega, kaunsa organ kis se connected hai. Dono bina ek dusre ke incomplete hain. Brain bina skeleton ke kuch nahi kar sakta, aur skeleton bina brain ke dead body hai.

Teeno (DSA + LLD + HLD) milke hi ek complete, scalable, real-world application ban paati hai.



Key Takeaways / Quick Recap

SUMMARY

1. **LLD** multiple DSA concepts ko combine karke ek complete application build karne ka process hai.
2. LLD sirf algorithms nahi — **objects, relationships, security, scalability, maintainability, reusability** par focus karta hai.
3. Teen core pillars hain: **Scalability** (millions of users sustain karna), **Maintainability** (easy debugging & feature addition), **Reusability** (plug-and-play modules).
4. Code **tightly coupled** nahi hona chahiye. Modules application-independent hone chahiye.
5. **HLD** tech stack, database, server scaling, aur cost optimization pe focus karta hai — isme negligible code hota hai.
6. **DSA = Brain | LLD = Skeleton | HLD = Environment/Architecture** — teeno milke complete system banta hai.

INTERVIEW FOLLOW-UP QUESTIONS

- "LLD mein scalability aur HLD mein server scaling mein kya farq hai?" (*Ans: LLD = code-level scalable design; HLD = infrastructure-level scaling*)
- "Tightly coupled code ko loosely coupled kaise banate hain?" (*Ans: Interfaces, abstraction, dependency injection — aage ke lectures mein cover karenge*)
- "Ek notification service ko reusable kaise design karoge?" (*Ans: Generic interface bana ke, specific app logic se alag rakh ke*)